Redis Architecture: Why Single-Thread Beats Multi-Thread

How Redis processes 1,00,000+ queries per second with a single thread and sub-millisecond latency.

⚡ 1. The 3 Speed Pillars

- **Pure In-Memory Storage:** All primary data resides in RAM. Zero disk seek latency (100ns RAM access vs 10ms HDD).
- **Non-Blocking I/O Multiplexing:** Uses Linux `epoll` (or BSD kqueue) to monitor 10,000+ client sockets in a single event loop.
- **Zero Lock Contention:** Single-threaded execution means zero mutexes, zero deadlocks, and zero thread context-switching CPU tax!

🏎️ 2. Linux epoll Mechanism

Traditional servers (e.g. Apache) spawn 1 thread per client. At 10,000 users, CPU spends 90% of time switching threads.

```
Client 1 ---\
Client 2 ---> [ epoll event queue ] -
> [ 1 Single Redis Thread ]
Client 3 ---/
Kernel alerts Redis ONLY when socket
has data ready.
```

Zero CPU cycles wasted polling idle connections.

🌲 3. Core Data Structures

- **SDS (Simple Dynamic String):** Pre-allocated buffer with O(1) length lookup (no C null-terminator scan).
- **Dict / Hash Table:** Incremental re-hashing over time so latency spikes never block queries.
- **Skiplist:** O(log N) sorted sets (ZSET) with lightweight pointer layers.

💻 4. Verification Benchmark

Verify Redis throughput on your local Linux machine:

```
# Run 100k requests with 50 parallel
clients
redis-benchmark -q -n 100000 -c 50 -t
set,get

# Typical output on modern quad-core:
SET: 114,285.71 requests per second
GET: 122,100.12 requests per second
```

All served by 1 CPU core!